

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	AB Dhanyashree	Reg. No.:	192472002
Section / Batch:	D/2024	Date:	28.08.26

Code of Conduct

I AB Dhanyashree(192472002) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

Algorithm / Pseudocode

Algorithm Reference_Resolution(Text)

1. Input the discourse.
2. Identify all named entities and pronouns.
3. For each pronoun:
 - a. Generate all possible antecedents.
 - b. Check gender agreement.
 - c. Check number agreement.
 - d. Check recency.
 - e. Check semantic compatibility.
 - f. Assign a score to each possible antecedent.
4. Select the antecedent with the highest valid score.
5. Replace each pronoun with its selected antecedent.
6. Return the resolved discourse.

End Algorithm

Apply to the Given Discourse Discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

1. Entities and Referring Expressions

Entities: Meena, Kavya, laptop, project.

Referring expressions:

- she
- one
- her
- She
- her

- it

2. Possible Antecedents

Referring Expression	Possible Antecedents
she	Meena, Kavya
one	laptop
her	Meena, Kavya
She	Meena, Kavya
her	Meena, Kavya
it	laptop

3. Constraint Analysis

- Gender: Meena and Kavya are both female, so gender alone cannot distinguish them.
- Number: Both are singular, so number also cannot distinguish them.
- Recency: Kavya is closer to “she”, but semantic context indicates that the person needing the laptop is likely Kavya.
- Semantic compatibility: “needed one for her project” strongly suggests Kavya needed the laptop.
- “She thanked her” → She = Kavya, her = Meena, because Kavya received the laptop and thanked Meena.
- “it” → laptop, because Kavya started using the laptop.

4. Final Resolution

- she → Kavya
- one → laptop
- her → Kavya
- She → Kavya
- her → Meena
- it → laptop

5. Resolved Discourse

“Meena gave Kavya a laptop because Kavya needed a laptop for Kavya’s project. Kavya thanked Meena and started using the laptop.”

How the Algorithm Avoids Incorrect Resolution

The algorithm does not depend only on grammatical properties. When both antecedents have the same gender and number, it uses recency, semantic compatibility, and discourse context. Therefore, even though both Meena and Kavya are female and singular, the context of needing a laptop for a project helps select the most appropriate antecedent.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

Algorithm Entity_Chain_Coherence(Text)

1. Divide the text into sentences.
 2. Identify entities and referring expressions.
 3. For each expression:
 - a. Find possible matching entities.
 - b. Compare semantic similarity and context.
 - c. Link the expression to the correct entity.
 4. Construct an entity chain for each entity.
 5. Count the number of sentences in which each entity appears.
 6. Calculate or estimate entity continuity.
 7. If important entities continue across sentences:
Mark discourse as coherent.
Else:
Mark discourse as less coherent.
 8. Return entity chains and coherence result.
- End Algorithm

Apply to the Given

Text Text: "Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

1. Entities and Referring Expressions

- Anjali
- laptop
- The laptop
- She
- it
- Anjali
- the computer
- processor
- Python
- machine learning application

2. Linking Expressions

Expression	Refers To
Anjali	Anjali
The laptop	laptop
She	Anjali
it	laptop
Anjali	Anjali
the computer	laptop

3. Entity Chains

Person chain: Anjali → She → Anjali

Device chain: laptop → The laptop → it → the computer

4. Entity Continuity

Sentence	Anjali Chain	Laptop Chain
S1	✓	✓
S2	–	✓
S3	✓	✓
S4	✓	✓

The laptop remains an important entity throughout the discourse, while Anjali also continues across the sentences.

5. Coherence Decision

The discourse is coherent because the main entities, Anjali and the laptop, are continuously maintained through repeated mentions and referring expressions.

Effect of Breaking an Entity Chain

If an entity chain is broken, the reader may not understand which entity a later expression refers to. For example, if “the computer” suddenly referred to an unrelated device, the connection between the sentences would become unclear. Therefore, maintaining entity chains improves continuity, clarity, and discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

Algorithm / Pseudocode

Algorithm Discourse_Relation_Identification(Text)

1. Divide the text into individual discourse units.
2. Identify explicit discourse markers.
3. For each pair of consecutive units:
 - a. Check for Cause–Effect relation.
 - b. Check for Sequence relation.
 - c. Check for Problem–Solution relation.
 - d. Check for Elaboration relation.
4. Analyze the contextual meaning.
5. Assign the most appropriate discourse relation.
6. Construct a discourse structure.
7. Return the identified relations.

End Algorithm

Apply to the Given Text

Text:

“The server crashed unexpectedly.”

“As a result, users could not access the application.”

“The technical team restarted the server.”

“After that, the system became available again.”

1. Discourse Units

DU1: Server crashed unexpectedly.

DU2: Users could not access the application.

DU3: Technical team restarted the server.

DU4: System became available again.

2. Discourse Markers

“As a result” → Cause–Effect

“After that” → Sequence

3. Semantic Relations

Units	Relation	Explanation
DU1 → DU2	Cause–Effect	Server crash caused users to lose access
DU2 → DU3	Problem–Solution	Access problem led to the technical team restarting the server
DU3 → DU4	Sequence	Restart happened before the system became available
DU3 → DU4	Cause–Effect	Restarting the server resulted in system availability

4. Discourse Structure

DU1: Server crashed

|

Cause

↓

DU2: Users could not access application

|

Problem

↓

DU3: Technical team restarted server

|

Solution / Cause

↓

DU4: System became available

5. Coherence Justification

The discourse is logically coherent because it follows a clear problem → effect → solution → result structure. The server crash causes the access problem, the technical team responds by restarting the server, and the system becomes available afterward. The markers “As a result” and “After that” make these relationships explicit.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

Algorithm / Pseudocode

Algorithm Dialogue_Act_Classification(Utterance)

1. Preprocess the utterance.
 2. Identify keywords, sentence structure, and speech patterns.
 3. Detect the communicative intention.
 4. If the utterance contains a greeting:
 Assign Greeting.
 5. Else if it asks for information:
 Assign Question.
 6. Else if it requests help/action:
 Assign Request.
 7. Else if it provides information:
 Assign Inform.
 8. Else if it offers help:
 Assign Offer.
 9. Else if it confirms information:
 Assign Confirmation.
 10. Return the dialogue act.
- End Algorithm

Apply to the Conversation

Speaker	Utterance	Dialogue Act	Reason
User	"Hello, I need help with my assignment."	Greeting + Request	"Hello" is a greeting and "I need help" requests assistance
Agent	"Sure, what problem are you facing?"	Question	Asks the user for information
User	"I do not understand machine learning."	Inform	Provides information about the user's problem
Agent	"I can explain the basic concepts to you."	Offer	Offers to explain machine learning

Dialogue-Act Sequence

Greeting + Request → Question → Inform → Offer

Importance in Conversational Agents

Dialogue-act recognition helps an agent understand what the user wants to achieve. For example, when the user makes a request, the agent can ask a relevant question; when the user provides information, the agent can use it to generate the next response. This makes the conversation coherent, relevant, and natural

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Algorithm / Pseudocode

Algorithm Dialogue_State_Tracking

1. Initialize all dialogue slots as empty:
 Departure City = NULL
 Destination City = NULL
 Travel Date = NULL
 Number of Passengers = NULL
 2. Read each user utterance.
 3. Extract information from the utterance.
 4. Match the information with the appropriate slot.
 5. Update the dialogue state.
 6. Check for missing slots.
 7. Select the next missing slot.
 8. Generate a question for that slot.
 9. Continue until all required information is collected.
 10. Return the completed dialogue state.
- End Algorithm

Initial Dialogue State

Slot	Value
Departure City	NULL
Destination City	NULL
Travel Date	NULL
Number of Passengers	NULL

Step-by-Step Application

User: "I want to book a flight."

No slot information is provided.

Slot	Value
Departure City	NULL
Destination City	NULL
Travel Date	NULL
Number of Passengers	NULL

Agent: "Where would you like to travel?"

User: “I want to go to Delhi.”

The system identifies Delhi as the destination city.

Updated State:

Slot	Value
Departure City	NULL
Destination City	Delhi
Travel Date	NULL
Number of Passengers	NULL

Agent: “From which city?”

User: “From Chennai.”

The system identifies Chennai as the departure city.

Final State After Given Conversation

Slot	Value
Departure City	Chennai
Destination City	Delhi
Travel Date	NULL
Number of Passengers	NULL

Next Appropriate Questions

Since Travel Date and Number of Passengers are still missing, the agent can ask:

“What date would you like to travel?”

After receiving the date, it can ask:

“How many passengers will be travelling?”

Importance of Dialogue State Tracking

Dialogue state tracking helps the conversational agent remember information provided in earlier turns. It prevents the agent from repeatedly asking for the same information and allows it to identify missing details. This improves coherence, consistency, and user experience in multi-turn conversations

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
KUMARAGURUBARAN T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____